

Invoice Processing Automation

Engineering Documentation: Architecture, Workflow & Design Decisions

Codebase: <https://github.com/msenzanje/invoice-processing-automation>

Project: Acme Corp AP Automation

June 2026

Table of Contents

1. Executive Summary	1
2. The Problem and the Approach	2
2.1 What was broken	3
2.2 The guiding principle: human-in-the-loop only where judgment is needed	3
2.3 Four principles that run through the whole codebase	3
3. System Architecture	3
3.1 High-level view	4
3.2 Technology stack.....	4
3.3 The shared state contract.....	5
4. The Workflow in Detail	5
4.1 The four agents.....	6
4.2 Worked example: how a single invoice flows	8
5. Project Structure	8
5.1 The three layers, and why they are separated	9
5.2 The files that matter most.....	10
6. Key Design Decisions and Their Trade-offs.....	10
6.1 LangGraph state machine over an imperative pipeline	11
6.2 Two-pass ingestion: deterministic first, LLM only as a fallback	11
6.3 Deterministic policy gate before the LLM	11
6.4 The self-critique reflection loop (draft → critique → revise).....	12
6.5 Fail-forward everywhere, never fail-hard.....	12
6.6 Side effects isolated behind tools/	12
6.7 Provider abstraction: Grok primary, Claude fallback.....	13
6.8 JSONL append-only audit log as the system of record	13
6.9 A read-only dashboard that never touches the pipeline.....	13

7. How the System Was Built (Development Timeline)	14
8. Testing and Quality	15
9. Running the System	16
9.1 Setup	17
9.2 Processing invoices	17
9.3 The dashboard	17
9.4 Configuration	17
10. Known Constraints and Future Work	18
11. Glossary.....	19

1. Executive Summary

Invoice Processing Automation (internal codename **Galatiq**) is a multi-agent AI system that replaces a manual accounts-payable (AP) workflow end to end. It ingests raw invoices in multiple formats, validates them against an inventory and vendor catalogue, makes a defensible approval decision, and then pays, rejects, or escalates each invoice, leaving a durable audit record behind every decision.

The system was built as a case study for **Acme Corp**, a PE-backed manufacturing firm losing an estimated **\$2M/year** to a fully manual process: invoices keyed by hand (≈30% error rate), validated against a legacy database, routed for VP approval through email threads, and paid via a banking API. There was no real audit trail and approvals lived in inboxes.

The central design idea is **selective autonomy**. Clean, low-value invoices from trusted vendors are approved automatically in milliseconds with no LLM call. Ambiguous or high-value invoices receive a careful, self-critiquing LLM review. Anything the system cannot confidently resolve is **escalated to a human** rather than guessed at. The system never silently auto-pays something it is unsure about.

Attribute	Value
Status	MVP : functional prototype, not production-deployed
Language	Python 3.10+
Orchestration	LangGraph state machine
Input formats	TXT, JSON, CSV, PDF (XML routed to LLM)
Interfaces	CLI (single + batch), FastAPI read-only dashboard
External integrations	All mocked (banking API, inventory DB seeded locally)
Test coverage	70 tests across all four agents; every LLM call mocked

2. The Problem and the Approach

2.1 What was broken

The manual AP process at Acme Corp had four recurring pain points:

- **Data entry error.** A ~30% error rate on hand-keyed extraction from messy PDFs.
- **No audit trail.** Approvals existed only as email chains — not searchable, not durable, and a compliance risk.
- **Staff burden.** Skilled finance staff spent their time on repetitive transcription.
- **Inconsistent fraud control.** Invalid or fraudulent invoices slipped through unevenly because checks depended on whoever was reviewing.

2.2 The guiding principle: human-in-the-loop *only where judgment is needed*

Rather than fully automate (risky) or merely assist a human (low leverage), the system draws a deliberate line:

- **Automate the boring, safe cases.** A clean invoice under \$1,000 from an approved vendor with high extraction confidence is approved automatically, with no model call.
- **Reason carefully about the ambiguous cases.** Anything flagged, high-value, or uncertain goes through a multi-step LLM review that critiques its own first draft.
- **Escalate the genuinely uncertain cases.** A third outcome `needs_review` exists precisely, so the system can hand off to a human instead of forcing a binary verdict.

2.3 Four principles that run through the whole codebase

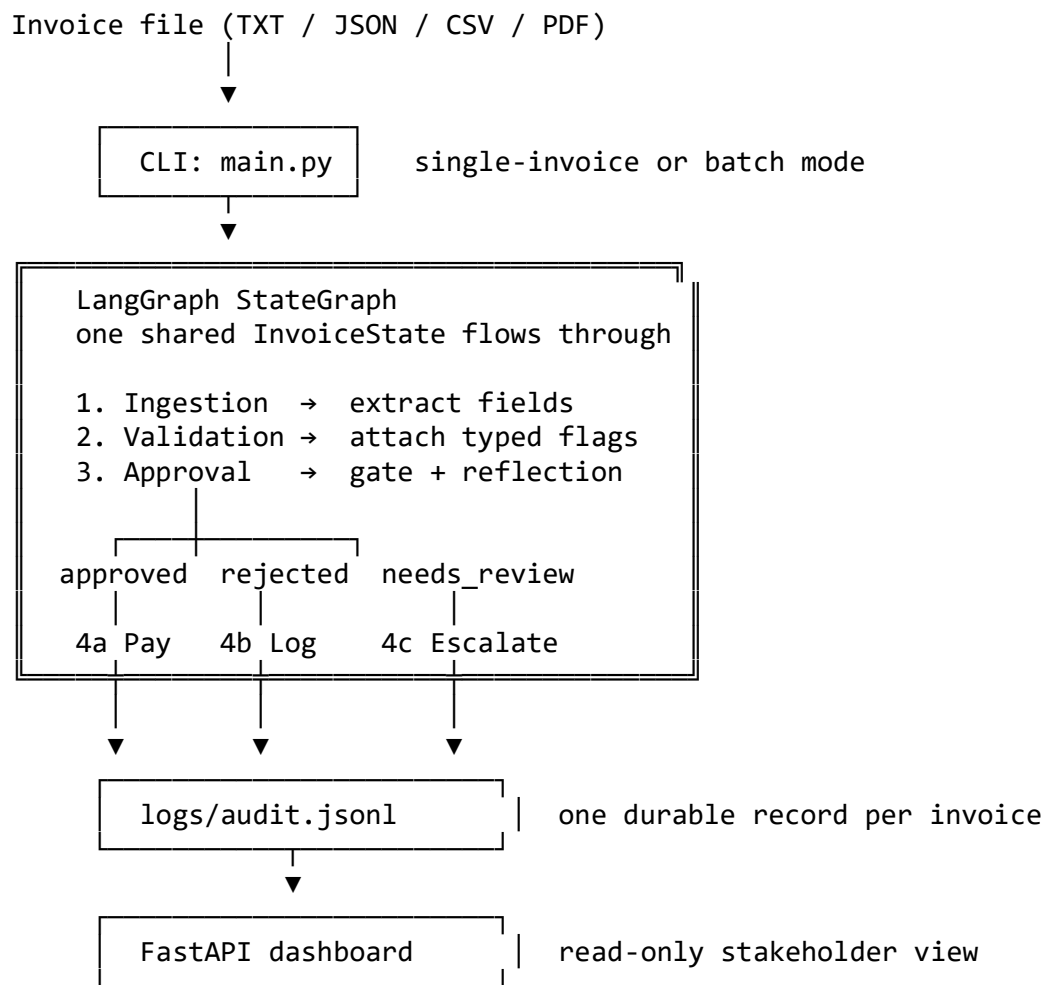
These appear repeatedly in the code and are worth internalizing before reading it:

1. **Flag, don't reject.** The validation stage *never* hard-fails an invoice. It attaches typed flags and passes them downstream. The approval agent is the *only* stage that renders a verdict.
2. **Fail-forward, never crash.** Every node takes state in and returns state out — it never raises into the graph. A failed extraction, an unreachable LLM, or a disk error routes the invoice to a safe terminal state (usually human review). One bad invoice never sinks a batch.
3. **Policy is code, not LLM judgment.** Hard rules (unrecognized vendors escalate; amount over \$10K → full review) are enforced *deterministically, before a token is ever spent*. A hallucinating model cannot override them.
4. **Admit uncertainty.** When the system is genuinely unsure, it escalates rather than coin-flipping.

3. System Architecture

3.1 High-level view

An invoice flows through **four sequential agents** and then **fans out** to one of three terminal nodes based on the approval decision. Each terminal node writes exactly one audit record.



The pipeline is **linear through approval**, then a **conditional edge** routes on the approval decision to one of three terminal nodes. All four stages read from and write to a single shared InvoiceState object.

3.2 Technology stack

Component	Choice	Role
Orchestration	LangGraph	Multi-agent state machine; nodes, edges, conditional routing
Data contracts	Pydantic v2	Typed models that cross every agent

Component	Choice	Role
		boundary
PDF extraction	pdfplumber (+ PyMuPDF fallback)	Primary text extraction; fallback when the text layer is sparse
CSV parsing	pandas	Tabular and key/value CSV layouts
LLM transport	httpx	Direct calls to the provider HTTP API
LLM providers	xAI Grok (primary), Anthropic Claude (fallback)	Reasoning for ingestion fallback and approval
CLI presentation	rich	Tables, progress bars, color output
Web dashboard	FastAPI + uvicorn	Read-only stakeholder view over the audit trail
Database	sqlite3 (stdlib)	Inventory and vendor catalogue
Config	python-dotenv	Environment-driven configuration
Tests	pytest	70 tests; all LLM calls and disk I/O mocked or temp-filed

The dependency list is deliberately lean. No heavyweight LLM-agent framework beyond LangGraph; no ORM; no front-end framework (the dashboard is plain HTML/CSS/JS).

3.3 The shared state contract

Every agent reads from and writes to a single `InvoiceState` (a `TypedDict` in `models/state.py`). As the invoice moves through the pipeline, state *accumulates*:

```
raw content → invoice_data → validation_result → approval_result → audit record
```

The `processing_stage` field tracks progress through the pipeline:

```
pending → extracted → validated → (approved | rejected | needs_review)
```

`InvoiceState` is the single most important contract in the project. **No data passes between agents via globals or function arguments, only through the state.** Adding a field to the workflow means declaring it here first. All fields have defaults so any node can be unit-tested in isolation.

4. The Workflow in Detail

4.1 The four agents

Agent 1 — Ingestion (`agents/ingestion.py`)

Turns a raw invoice file into a structured, confidence-scored `InvoiceData` object using a **two-pass design**:

- **Pass 1 — deterministic (fast, zero cost).** Format-specific extractors: `pdfplumber/PyMuPDF` for PDF, compiled regexes for TXT, `pandas` for CSV, `json` for JSON. Clean matches earn **HIGH** confidence; fuzzy fallbacks earn **MEDIUM**.
- **Pass 2 — LLM (only when needed).** If Pass 1 leaves a required field missing or low-confidence, the raw text is handed to the LLM under a strict Pydantic schema. A **self-correcting retry loop** re-prompts the model with the validation error appended on failure, up to 3 attempts. LLM results are *merged* into the Pass-1 results, so a HIGH-confidence deterministic field is never overwritten by a model that might hallucinate.

A clean invoice never pays an "LLM tax" — it is extracted entirely by Pass 1. If Pass 2 exhausts its retries, ingestion raises a `ValueError` carrying every failed attempt, and the graph routes the invoice to human review.

Each field carries a per-field confidence rating (HIGH / MEDIUM / LOW). An overall confidence is computed by majority rule (HIGH if 3+ required fields are HIGH; LOW if 2+ are LOW; otherwise MEDIUM). **Confidence flows forward** — the approval agent weighs it rather than re-deriving it.

Agent 2 — Validation (`agents/validation.py`)

Checks the extracted data against `inventory.db` and emits a typed `ValidationResult`. Its philosophy is **flag, don't reject**: even a heavily-flagged invoice produces a clean result object for the approval agent to reason over.

Flag code	Scope	Severity	Meaning
UNKNOWN_ITEM	item	error	Item not in the inventory catalogue
STOCK_EXCEEDED	item	error	Requested quantity exceeds available stock
ZERO_STOCK	item	error	Item exists but has zero stock
NEGATIVE_QUANTITY	item	error	Line-item quantity is negative
VENDOR_UNRECOGNIZED	invoice	warning	Vendor not on the approved-vendor list
ZERO_AMOUNT	invoice	warning	Invoice total is zero or negative
LOW_CONFIDENCE_EXTRACTION	invoice	warning	A required field was extracted with low trust

The flag taxonomy and its severities are a **fixed contract**, expressed as enums (not free strings) because the approval agent routes on flag severity. Note, there is deliberately **no "fraud" flag**, no single check can assert fraud, so rejection is never a deterministic verdict; it can only emerge from the approval loop.

Agent 3 — Approval (`agents/approval.py`)

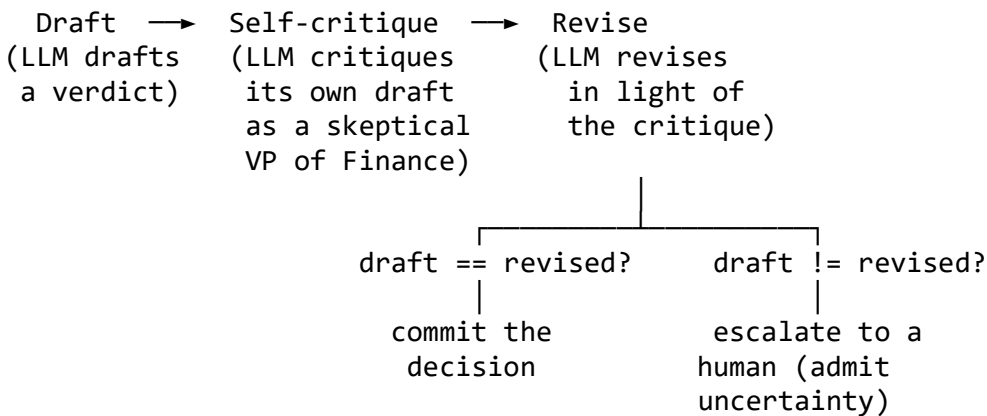
The most agentic part of the pipeline, with **two layers**.

Layer 1 — a deterministic gate applies hard policy rules *before* spending a token, in strict priority order:

1. Unrecognised vendor → **always escalate** to human review (no LLM).
2. Any error-severity flag → **full critique loop**.
3. Amount over \$10,000 → **full critique loop**.
4. Clean, under \$1,000, high-confidence → **fast-track approve** (no LLM).
5. Everything else (warning-only, mid-range amounts) → **full critique loop**.

Priority matters: the vendor rule is checked *first*, so an unrecognised vendor always lands in human review even on an otherwise loop-worthy invoice.

Layer 2 — a reflection loop runs only for invoices the gate routes to FULL_LOOP:



The critique step explicitly prompts the model to play a **skeptical VP of Finance** and surface risks the draft under-weighted. If the draft and revision **agree**, that decision is committed. If they **disagree**, the invoice escalates to a human with both passes and the critique surfaced. If the LLM is unreachable, the result escalates to `needs_review`, never a crash, never a silent auto-approve. The full draft → critique → revise transcript is captured for the audit log.

Agent 4 — Terminal nodes (`agents/payment.py`)

Three terminal nodes, one per approval outcome, each writing **exactly one** audit record:

- **payment_node** (approved) → calls `mock_payment`, records the result with a `payment_status`.
- **log_node** (rejected, or a dead upstream invoice) → records the rejection and its reasoning.
- **escalation_node** (needs_review) → records the invoice plus its reflection transcript for the human-review queue.

Audit-record assembly is shared across all three so the schema can never drift into three copies. The audit write is **swallow-but-signal**: an I/O error never crashes a payment that already succeeded, it is logged into `processing_log` so an operator can see the durable record didn't land.

4.2 Worked example: how a single invoice flows

Consider `invoice_1005.json` — a request for 8 units of GadgetX, of which only 5 are in stock:

1. **Ingestion** parses the JSON deterministically (Pass 1, HIGH confidence) — no LLM call.
2. **Validation** looks up GadgetX, finds stock = 5, and raises `STOCK_EXCEEDED` (error-severity).
3. **Approval gate** sees an error-severity flag → routes to `FULL_LOOP`.
4. **Reflection loop** drafts a decision, critiques it from a VP stance ("why is this quantity 60% over stock?"), and revises. The final verdict (e.g. `rejected` or `needs_review`) is committed if draft and revision agree, else escalated.
5. **Terminal node** writes one audit record with the decision, reasoning, critique, and flags.

A clean invoice (`invoice_1001.txt`, small amount, approved vendor) skips steps 3–4's LLM work entirely: the gate fast-tracks it and `payment_node` records an approved, paid record in milliseconds.

5. Project Structure

invoice-processing-automation/

— main.py	# CLI entry point – single-invoice & batch modes
— setup_db.py	# One-shot seeder for inventory.db (run first)
— requirements.txt	
— .env.example	# Copy to .env and add your API key
— agents/	# The four pipeline stages
— ingestion.py	# 1 · extract structured data (two-pass)
— validation.py	# 2 · check against catalogue, attach flags
— approval.py	# 3 · deterministic gate + reflection loop
— payment.py	# 4 · payment / rejection-log / escalation
— graph/	
— workflow.py	# LangGraph state machine: nodes, edges, routing
— models/	# Pydantic / TypedDict data contracts
— state.py	# InvoiceState – the shared state object
— invoice_data.py	# InvoiceData, LineItem, Confidence
— results.py	# ValidationResult, ApprovalResult,
AuditRecord	
— tools/	# Side-effect boundaries (isolated, swappable)
— llm.py	# the ONLY module that talks to a provider API
— db.py	# SQLite read helpers + schema
— audit.py	# the one place that appends to audit.jsonl
— payment_api.py	# mock payment gateway
— dashboard/	# FastAPI read-only stakeholder view
— app.py	# server: serves the page + /api/summary
— audit_source.py	# reads audit.jsonl → dashboard payload
— templates/index.html	
— static/	# styles.css + app.js (no framework)
— data/invoices/	# 20 sample invoices across all formats
— logs/audit.jsonl	# the durable audit trail (created on first run)
— tests/	# pytest suites, all LLM calls mocked, no live
I/O	
— test_ingestion.py	
— test_validation.py	
— test_approval.py	
— test_payment.py	

5.1 The three layers, and why they are separated

The directory layout encodes the core architectural discipline of the project:

- **agents/ — pure logic.** Each agent is a pure function: `state in → state out`, never raises. No I/O lives here directly.
- **tools/ — side-effect boundaries.** Every side effect (LLM calls, DB reads, disk writes, payments) is isolated behind a small, swappable module. This is what keeps the agents testable and the I/O boundaries explicit.
- **models/ — typed contracts.** The interfaces *between* agents are Pydantic models. Flag codes and decisions are enums, not strings.

5.2 The files that matter most

File	Why it matters
<code>models/state.py</code>	The shared <code>InvoiceState</code> contract. A change here touches all four agents.
<code>graph/workflow.py</code>	The single source of truth for workflow logic and routing. Business-rule changes (e.g. raising a threshold) start here or in the approval gate.
<code>agents/approval.py</code>	The deterministic gate + reflection loop — the heart of the system's "judgment".
<code>tools/llm.py</code>	The only module that imports a provider SDK/endpoint. Switching providers is a one-file change.
<code>tools/audit.py</code>	The one place that appends to the legally-significant audit trail.

6. Key Design Decisions and Their Trade-offs

This section is the core of the documentation for a developer audience: *what was chosen, why, and what was given up*.

6.1 LangGraph state machine over an imperative pipeline

Decision. Model the workflow as a LangGraph StateGraph — nodes, edges, and a conditional edge after approval — rather than a hand-written sequence of function calls.

Why. The workflow is a state machine: linear through approval, then a three-way fan-out. LangGraph makes the routing *declarative* and the fan-out explicit. Adding a stage is "add a node, wire two edges". The graph definition becomes self-documenting.

Trade-off. A framework dependency and a learning curve for anyone new to LangGraph; for a strictly linear pipeline, plain functions would be lighter. The fan-out and the "pure node" discipline justify the cost here. (*This choice was fixed early and intentionally not revisited*).

6.2 Two-pass ingestion: deterministic first, LLM only as a fallback

Decision. Always attempt deterministic, format-specific extraction first. Call the LLM only when a required field is missing or low-confidence. Even then, *merge* rather than *replace*.

Why.

- **Cost and latency.** A clean invoice costs zero LLM calls and resolves in milliseconds. The LLM is a fallback, not the default path.
- **Trust.** Deterministic regex/JSON parsing is reproducible and auditable. HIGH-confidence Pass-1 fields are never overwritten by a model that could hallucinate.
- **Robustness.** The self-correcting retry loop (re-prompt with the validation error appended) recovers from transient malformed-JSON responses.

Trade-off. The deterministic extractors are non-trivial to write and maintain — there are carefully tuned regexes for TXT and two CSV layouts (columnar and vertical key/value). That maintenance burden is the price of not paying an "LLM tax" on every clean invoice and of keeping extraction auditable.

6.3 Deterministic policy gate *before* the LLM

Decision. Hard business rules (vendor escalation, \$10K threshold, error-flag handling, fast-track) are enforced in pure Python *before* any model call. The LLM is consulted only for FULL_LOOP cases.

Why. Policy must be **non-negotiable and inspectable**. A model cannot be allowed to approve an unrecognised vendor or wave through a \$50K invoice because it was "convinced". Putting policy in code means it is unit-testable (eight gate tests cover every

branch and priority interaction) and a hallucination can never override it. It is also cheaper: invoices that policy can resolve never spend a token.

Trade-off. The thresholds (\$1,000, \$10,000) are coarse business judgments hard-coded as defaults. They are env-overridable, but they are not learned or adaptive. That is the correct trade-off for an auditable financial system. This means policy tuning is a human decision.

6.4 The self-critique reflection loop (draft → critique → revise)

Decision. For ambiguous invoices, the LLM drafts a decision, then critiques its *own* draft from an adversarial "skeptical VP" stance, then revises. If draft and revision disagree, the system escalates.

Why. A single LLM pass is prone to anchoring on a first impression. Forcing an explicit adversarial critique surfaces under-weighted risks and produces a **defensible, logged rationale** for every contested decision, directly replacing the human deliberation that used to happen in email. Disagreement between passes is treated as a genuine signal of uncertainty, which is exactly when a human should look.

Trade-off. The loop costs **three LLM calls** per ambiguous invoice (draft, critique, revise), so a contested invoice takes several seconds and 3× the tokens of a single pass. That cost is accepted because (a) the loop only runs for invoices that truly do need scrutiny; clean ones are fast-tracked, and (b) for an AP system, a well-reasoned, auditable decision is worth far more than raw throughput. Correctness is explicitly the top priority over speed.

6.5 Fail-forward everywhere, never fail-hard

Decision. No graph node ever raises into the orchestrator. Every failure mode (failed extraction, DB error, unreachable LLM, failed audit write) routes to a safe terminal state and still produces an audit record.

Why. Two reasons. First, **batch resilience**, one malformed invoice should never sink a 1,000-invoice run. Second, **safe defaults**, when in doubt, escalate to a human; never crash, and never silently auto-pay. The router itself uses a `.get(..., escalate)` default so even an unmapped future decision fails *toward* human review rather than raising a `KeyError`. (*This safety property was hardened explicitly during development — see §7.*)

Trade-off. Failures are quieter, so a swallowed audit-write error becomes a log line rather than a stack trace, so observability has to be deliberate (hence the `processing_log` "swallow-but-signal" pattern and `DEBUG` logging around LLM calls). The alternative, crashing loudly is unacceptable in a batch financial pipeline, so we pay for safety with a reliance on logging.

6.6 Side effects isolated behind `tools/`

Decision. Every side effect lives behind a small module in `tools/`: `llm.py` (the only provider-aware code), `db.py`, `audit.py`, `payment_api.py`. Agents call these; they never perform I/O inline.

Why. This is what makes the agents pure and therefore testable, and it makes each integration **swappable in one place**. Switching LLM providers, redirecting the audit log, or replacing the mock payment gateway with a real banking API is a single-file change. The test suite exploits this directly, it injects `MockLLMClient` and points DB/audit helpers at temp files, so the entire suite runs with zero network calls and zero writes to the real database or log.

Trade-off. A little indirection: reading the approval flow means also opening `tools/llm.py`. The payoff is testability and swappability. This is a deliberate, repeated pattern.

6.7 Provider abstraction: Grok primary, Claude fallback

Decision. `tools/llm.py` exposes a single `complete(prompt) -> str` protocol. xAI Grok is the primary provider; Anthropic Claude is the fallback, auto-selected from whichever API key is present. Both are reached over plain `httpx`, with `temperature=0` for determinism.

Why. Agents have no idea which provider is behind a call. Auto-detection from the configured key makes local setup frictionless, and `temperature=0` keeps decisions as reproducible as an LLM allows. Using `httpx` directly (rather than each vendor's SDK) keeps the dependency surface small and the two code paths symmetric.

6.8 JSONL append-only audit log as the system of record

Decision. Every invoice leaves exactly one JSON object appended to `logs/audit.jsonl`. The file is **only ever appended**, never truncated. The schema is a fixed contract: `invoice_id`, `timestamp`, `vendor`, `amount`, `flags`, `decision`, `reasoning`, `critique`, `payment_status`, `processing_time_ms`.

Why. This file *is* the replacement for the VP email-approval chain and can be treated as legally significant. Append-only JSONL is simple, human-readable, trivially streamable, and **tamper-evident by construction**. You cannot quietly edit a past decision. `timestamp` (the processing *start*) plus `processing_time_ms` reconstructs the full processing window from a single record. The schema is shared by all three terminal nodes so it cannot drift.

Trade-off. JSONL is not a queryable database — large-scale analytics would eventually want the records loaded into a real store. For an auditable MVP, simplicity and tamper-evidence outweigh queryability, and the dashboard's read model already shows the file can be projected into views cheaply.

6.9 A read-only dashboard that never touches the pipeline

Decision. The FastAPI dashboard is strictly read-only. It reads `logs/audit.jsonl` through its own read model (`dashboard/audit_source.py`) and never imports the graph or the agents. The frontend is plain HTML/CSS/JS with no framework and polls a single `/api/summary` endpoint.

Why. A clean separation between *producing* decisions (the pipeline) and *viewing* them (the dashboard). The dashboard can never accidentally mutate a decision or re-run an agent. A single endpoint that recomputes everything from one file read avoids three reads per refresh, and embedding the first payload at page load avoids an empty-state flash. No front-end framework keeps the dependency list lean and the page cache-friendly.

Trade-off. Updates arrive on a short delay rather than instantly. For a stakeholder demo view, the simplicity is worth the small latency.

7. How the System Was Built (Development Timeline)

The project was built in clearly scoped phases, each landed as a reviewed pull request. The phasing follows the pipeline itself, each stage was built and tested before the next consumed it.

Phase	Scope	Outcome
0 — Scaffold	Project skeleton, InvoiceState, package layout	The shared contract and structure other phases build on
1 — Ingestion	Two-pass extraction, four formats, confidence scoring, self-correcting LLM fallback	Raw files → typed InvoiceData
2 — Validation	Inventory/vendor checks, the typed flag taxonomy	InvoiceData → ValidationResult (flag, don't reject)
3 — Approval	Deterministic gate + draft→critique→revise reflection loop	ValidationResult → committed ApprovalResult
4 — Payment & audit	Three terminal nodes, the JSONL audit record, mock payment	One durable record per invoice on every branch
5 — Robustness fix	Unmapped-decision routing hardened to escalate rather than crash	Closed a fail-forward gap in the router (<code>.get(..., escalate)</code>)
6 — CLI	--batch directory processing, richer rich terminal UI, per-invoice timing	Operator-facing single + batch modes
7 — Dashboard	FastAPI read-only stakeholder view over the audit trail	Live, filterable view of every decision
8 — Hardening & docs	.gitignore, .env.example, README	Repo made shareable and reproducible

Two things are worth calling out for the team:

- **The contract-first ordering paid off.** Because InvoiceState and the Pydantic result models were defined before the agents that produce them, each agent could be built and unit-tested against a stable interface, with the next stage mocking what it consumed.
- **The robustness fix (Phase 5) is a good illustration of the fail-forward principle in practice.** The router originally indexed a decision→node map directly; a decision not in the map would have raised a KeyError into the graph. It was changed to route unmapped decisions to human review, keeping the "never raises" contract intact even against future enum additions.

8. Testing and Quality

The suite contains **70 tests across the four agents**, and is built on two rules that make it fast and safe to run anywhere:

- **No live LLM calls.** Every model interaction is driven by `MockLLMClient`, which can script successive responses, e.g. invalid JSON twice then valid JSON, to exercise the self-correction loop.
- **No live I/O.** DB and audit tests run against temporary files (`tmp_path`), so nothing ever touches the real `inventory.db` or `logs/audit.jsonl`.

What the suite covers:

Area	Representative coverage
Ingestion	Clean extraction per format; the Pass-1→Pass-2 confidence gate; self-correction recovering on the third attempt; exhausted retries raising with full context; HIGH-confidence merge precedence; the overall-confidence majority rule
Validation	Every flag (<code>UNKNOWN_ITEM</code> , <code>STOCK_EXCEEDED</code> , <code>ZERO_STOCK</code> , <code>NEGATIVE_QUANTITY</code> , <code>VENDOR_UNRECOGNIZED</code> , <code>ZERO_AMOUNT</code> , <code>LOW_CONFIDENCE_EXTRACTION</code>); <code>ZERO_STOCK</code> superseding <code>STOCK_EXCEEDED</code> ; blank vendor handling; a heavily-flagged invoice still not raising; DB-error fail-forward
Approval	All four gate routes and their priority interactions (vendor rule beating error/threshold); the loop making exactly three calls and capturing the transcript; self-correction on malformed draft JSON; agree→commit and disagree→escalate resolution; LLM transport failure failing forward to review; fast-track never calling the LLM
Payment / audit	Each terminal node writing the correct record; critique null when the loop didn't run; <code>payment_status</code> only on the approved path; the dead-invoice path writing a rejection from <code>error_message</code> ; the writer appending (not overwriting); a write failure returning an error string instead of raising

The emphasis throughout is on the **decision boundaries and the failure modes**, the gate's priority order, the reflection loop's agree/disagree split, and every fail-forward guard, because on an AP system a wrong decision (especially a wrong *approve*) is the expensive failure.

Run the suite with:

```
pytest                # everything
pytest -v             # verbose
pytest tests/test_approval.py  # one suite
```

9. Running the System

9.1 Setup

```
git clone https://github.com/msenzanje/invoice-processing-automation.git
cd invoice-processing-automation
```

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

```
pip install -r requirements.txt
```

```
cp .env.example .env              # then set XAI_API_KEY=...
python setup_db.py                # seeds inventory.db (idempotent)
```

setup_db.py is idempotent and prints the seeded inventory and vendors so you can see exactly what invoices are validated against.

9.2 Processing invoices

Single invoice, prints fields, flags, decision, and the reflection transcript

```
python main.py --invoice_path=data/invoices/invoice_1001.txt
```

Same, with DEBUG-level logging (LLM reasoning, gate routing)

```
python main.py --invoice_path=data/invoices/invoice_1001.txt --verbose
```

Whole directory, live progress bar + a paid/rejected/review summary

```
python main.py --batch --invoice_dir=data/invoices/
```

Exactly one of --invoice_path or --batch must be provided. Batch mode is fail-forward: a crash on one invoice records a synthetic failed state and the run continues.

9.3 The dashboard

```
uvicorn dashboard.app:app --reload    # or: python -m dashboard
```

Then open <http://127.0.0.1:8000>. The page shows executive-summary KPIs (invoices processed, auto-approval rate, average processing time, total value approved), a searchable/filterable table with each row expandable to its reasoning, critique, and flags, and a live feed that animates new records in as the pipeline appends them. Run the pipeline in one terminal and watch invoices appear in the browser.

9.4 Configuration

All configuration is environment-driven via .env:

Variable	Default	Purpose
XAI_API_KEY	—	xAI Grok key (primary provider)

Variable	Default	Purpose
ANTHROPIC_API_KEY	—	Anthropic Claude key (fallback)
LLM_PROVIDER	auto	Force grok or claude; auto-detects from the key present
XAI_MODEL	grok-3	Grok model name
LLM_TIMEOUT	60	LLM request timeout (seconds)
APPROVAL_THRESHOLD_HIGH	10000	Amount above which the full critique loop always runs
APPROVAL_THRESHOLD_LOW	1000	Amount below which clean invoices fast-track
LOG_LEVEL	INFO	Logging level

As discussed above, changing approval policy is a one-line env override, no code change. Switching providers is a one-line change in `tools/llm.py`.

10. Known Constraints and Future Work

This is an MVP. The boundaries are deliberate.

Current constraints

- **All external integrations are mocked.** No real banking API, email inbox, or production inventory database. `mock_payment` returns a deterministic success envelope shaped like a real gateway response.
- **Performance is the lowest priority for the MVP.** A contested invoice needs 2–3 LLM calls at ~1–2 s each, so 3–8 s end to end. Batch processing is sequential by default.
- **The audit log is a flat file**, not yet a queryable store.
- **Exact item-name matching.** Inventory lookups are exact (the item is the primary key); fuzzy item matching is left to ingestion-side normalization.

Natural next steps

- Project the JSONL trail into a queryable database for analytics, keeping the append-only file as the source of record.
- Add fuzzy item/vendor matching with a human-confirmation step.

11. Glossary

Term	Meaning
InvoiceState	The shared TypedDict every agent reads from and writes to as the invoice flows through the pipeline.
Pass 1 / Pass 2	Deterministic extraction (Pass 1) and the LLM fallback (Pass 2) in the ingestion agent.
Flag	A typed validation finding with a code, a severity (warning/error), and a message.
The gate	The deterministic policy layer in the approval agent that routes an invoice <i>before</i> any LLM call.
Reflection loop	The draft → critique → revise LLM cycle for ambiguous invoices.
FULL_LOOP / FAST_APPROVE / AUTO_REVIEW	The three routes the gate can choose. Only FULL_LOOP calls the LLM.
needs_review	The third approval outcome — escalate to a human rather than approve or reject.
Fail-forward	The principle that a node never raises; failures route to a safe terminal state with an audit record.
Audit record	The single JSON line written to <code>logs/audit.jsonl</code> per invoice — the system of record.
Swallow-but-signal	An I/O error is not raised but is recorded in <code>processing_log</code> so it remains visible.